



**FACULTY OF TECHNOLOGY AND ENGINEERING
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY
AND RESEARCH**

DEPARTMENT OF INFORMATION TECHNOLOGY

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms



PRACTICAL-3

CODE :

<https://github.com/abhishekipatadiya28-hash/DATA-STRUCTUREANDALGORITHM/tree/main/Practical-3>

PROBLEM-1

DIFFICULTY:

I found it difficult to understand the three sorting methods: Bubble Sort, Selection Sort, and Insertion Sort. I was confused about how each method works.

ERRORS:

I made some mistakes while writing the loops. Sometimes the elements were not sorted properly. I was also confused about when to swap the elements.

SOLUTION:

First, I understood the basic working of all three sorting methods.

- Bubble Sort: Compares nearby elements and swaps them if they are in the wrong order.
- Selection Sort: Finds the smallest element and places it in the correct position.
- Insertion Sort: Takes one element at a time and places it in the correct position in the sorted part.

After understanding these methods, I wrote the code and successfully sorted the marks.

If the data is already sorted, Insertion Sort can be faster because it requires very few changes.

OUTPUT:

```

please say key to continue
process returned 0 (0x0)   execution time : 15'818 s
50 51 20 02 80
insertion
80
02
20
51
50
result
80
02
20
51
50
selection
Enter marks of student 2 51
Enter marks of student 1 80
Enter marks of student 3 02
Enter marks of student 5 50
Enter marks of student 7 20
Enter the number of students :2

```

OUTCOMES:

- Learned the basic working of three sorting methods.
- Understood the difference between the three methods.
- Learned how to sort marks using different methods.
- Understood that Insertion Sort is useful for already sorted data.

PROBLEM-2 DIFFICULTY:

The main difficulty was arranging 0, 1, and 2 in the correct order without using another array.

CSUC201

25DIT045

ERRORS:

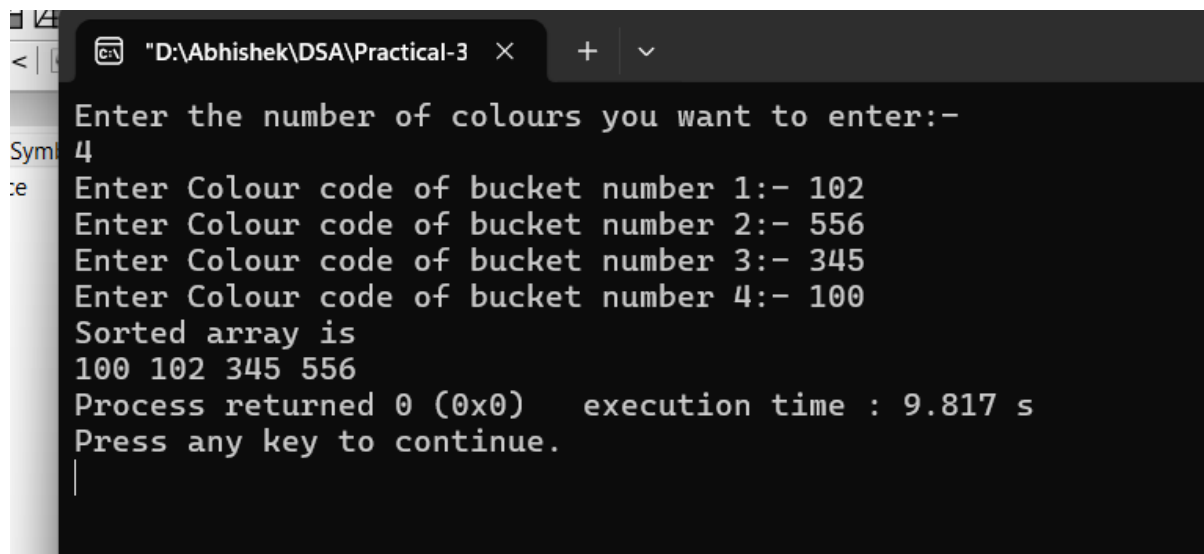
Initially, I thought of using another array, but the question did not allow extra storage. I was also confused about how to move 0s and 2s to their correct positions.

SOLUTION:

I used the same array to arrange the values. I kept 0s at the beginning, 1s in the middle, and 2s at the end.

I used different positions to keep track of where 0 and 2 should be placed. This method can arrange the complete array in one pass.

OUTPUT:



```
"D:\Abhishek\DSA\Practical-3"
Enter the number of colours you want to enter:-
4
Enter Colour code of bucket number 1:- 102
Enter Colour code of bucket number 2:- 556
Enter Colour code of bucket number 3:- 345
Enter Colour code of bucket number 4:- 100
Sorted array is
100 102 345 556
Process returned 0 (0x0)    execution time : 9.817 s
Press any key to continue.
```

OUTCOMES:

- Learned how to arrange 0, 1, and 2.
- Understood how to sort an array without using another array.
- Learned the one-pass approach.
- Understood that one-pass sorting can be more efficient.

KEY QUESTION:-

1. [Problem 1] One of the three methods can detect early that the list is already sorted and stop without doing any more work. Which method is it, how would you add that detection, and why can the other two not do the same?

ANSWER:-

Bubble Sort can be modified to stop early by using a swapped variable. If no swap occurs during a complete pass, the list is already sorted, so the algorithm stops. Selection Sort cannot stop early because it still needs to search for the minimum element in the remaining part.

2. [Problem 2] If your solution makes two passes through the list, can you identify which decision in your logic forces the second pass? Is there a way to restructure that decision to eliminate it?

ANSWER:-

The second pass is caused by separating the counting and rearranging operations. We can eliminate it by rearranging elements during the same traversal using three pointers. This allows the problem to be solved in one pass with $O(n)$ time and $O(1)$ extra space.